



Bachelorarbeit

Systematische Transformation KI-generierter
Web-Architekturen in sichere, lokale
Desktop-Anwendungen unter Anwendung des
Local-First-Paradigmas

Joshua Ohans

Prüfer: Prof. Dr. Matthias Teßmann
Prof. Dr. Ronald Petrlc

© Joshua Ohans, 2026

Diese Arbeit, einschließlich ihrer Teile, **ist urheberrechtlich geschützt**. Jede Verwertung außerhalb der engen Grenzen des Urheberrechtsgesetzes ohne Zustimmung von dem Urheber ist untersagt und strafbar. Dies gilt insbesondere für Vervielfältigungen, Übersetzungen, Mikroverfilmung und die Speicherung und Verarbeitung in elektronischen Systemen.

Danksagung

Only if you want to thank someone.

Kurzdarstellung

Kurze Zusammenfassung der Arbeit, höchstens halbe Seite. Deutsche Fassung auch nötig, wenn die Arbeit auf Englisch angefertigt wird.

Schlagwörter

KI-Prototyp, Local-First, Desktop-Anwendung, Sicherheit, Elektron, Transformation

Inhaltsverzeichnis

1	Einleitung	5
2	Theoretischer Hintergrund	6
2.1	KI-generierte Software: Werkzeuge, Qualität und Grenzen	6
2.1.1	Überblick über KI-gestützte Entwicklungstools	6
2.1.2	Charakteristika und Qualitätsdefizite KI-generierten Codes	7
2.2	Transformation von Web-Anwendungen in Desktop-Anwendungen	8
2.2.1	Referenzarchitektur KI-generierter Web-Anwendungen	9
2.2.2	Desktop-Runtimes: Electron als Integrationsruntime für Web-Anwendungen . .	11
2.2.3	Sicherheitsmodell von Electron	13
2.3	Sicherheit lokaler Desktop-Anwendungen	16
2.3.1	Bedrohungsmodell und Angriffsflächen	16
2.3.2	Sicherheitsrisiken KI-generierter Anwendungen	16
2.3.3	Sicherheitsanforderungen an das Zielartefakt	17
2.4	Das Local-First-Paradigma	18
2.4.1	Kernprinzipien nach Kleppmann et al.	18
2.4.2	Lokale Datenhaltung und Offline-First	19
2.4.3	Abgrenzung zu cloudzentrierten und traditionellen Desktop-Anwendungen . . .	20
2.4.4	Relevanz für das Transformationsszenario	21
3	Methodisches Vorgehen	23
4	Analyse und Problemanalyse	24
5	Entwurf und Entwicklung des Artefakts	25
6	Evaluation und Demonstration	26
7	Diskussion	27
8	Fazit und Ausblick	28
A	Supplemental Information	34

Abkürzungsverzeichnis

BaaS Backend-as-a-Service

CWE Common Weakness Enumeration

IDE Integrated Development Environment

LLM Large Language Model

SPA Single-Page Application

Kapitel 1

Einleitung

Kapitel 2

Theoretischer Hintergrund

2.1 KI-generierte Software: Werkzeuge, Qualität und Grenzen

Die vorliegende Arbeit setzt bei einem KI-generierten Web-Prototyp an, dessen Entstehung und Charakteristika für das gesamte Transformationsvorhaben grundlegend sind. Dieser Abschnitt ordnet zunächst die Landschaft der KI-gestützten Entwicklungswerkzeuge ein, aus der ein solcher Prototyp typischerweise hervorgeht. Anschließend werden die spezifischen Qualitätsdefizite herausgearbeitet, die den mit diesen Werkzeugen erzeugten Code auszeichnen, und deren Konsequenzen für eine sichere Weiterverwendung erörtert.

2.1.1 Überblick über KI-gestützte Entwicklungstools

Seit der Einführung leistungsfähiger Large Language Models (LLMs) hat sich eine Praxis etabliert, bei der Software überwiegend durch natürlichsprachliche Prompts anstelle manuell geschriebenen Codes entsteht. Der von Andrej Karpathy geprägte Begriff *Vibe Coding* beschreibt diese Praxis treffend als ein Vorgehen, bei dem Entwicklerinnen und Entwickler den generierten Code akzeptieren, ohne ihn im Detail zu prüfen, und sich stattdessen auf das beobachtbare Verhalten der Anwendung verlassen [1]. Diese Arbeitsweise senkt die Einstiegshürde für die Softwareentwicklung erheblich, da auch technische Laien funktionsfähige Prototypen erstellen können [2]. Aus dieser Entwicklung ist eine heterogene Landschaft spezialisierter Werkzeuge entstanden, die sich hinsichtlich Zielsetzung, bevorzugtem Technologie-Stack und Grad der Cloud-Abhängigkeit deutlich unterscheiden.

Replit positioniert sich als Online-Integrated Development Environment (IDE) mit integriertem KI-Agenten und fokussiert sich auf vollständige Full-Stack-Web-Prototypen, deren Persistenzschicht standardmäßig an eine cloudseitige Datenbank beziehungsweise einen verwalteten Datenbankdienst gekoppelt ist und deren Ausführung in der Replit-Cloud erfolgt [3]. Diese Architekturentscheidung begünstigt zwar schnelle, sofort ausführbare Prototypen, erzeugt jedoch eine starke Abhängigkeit von der Plattform und erschwert eine native lokale Ausführung. **Lovable** spezialisiert sich auf die Generierung vollständiger Anwendungen aus natürlicher Sprache und verknüpft den erzeugten Code standardmäßig mit einem Backend-as-a-Service (BaaS)-Anbieter, in der Regel Supabase, für Authentifizierung und Datenhaltung [4]. Dieses Kopplungsmuster beschleunigt die Prototyp-Erstellung, verlagert die Verantwortung für eine korrekte Sicherheitskonfiguration jedoch weitgehend auf die Anwendenden. **Cursor** und **GitHub Copilot** verfolgen demgegenüber einen assistenzbasierten Ansatz. Sie sind in einen bestehenden Entwicklungsworkflow eingebettet und ergänzen dort die Codeerstellung zeilen- oder funktionsweise, stellen jedoch kein eigenständiges Prototyping-Werkzeug für vollständige Anwendungen dar. **Figma Make** adressiert wiederum die Übersetzung von Design-Artefakten in Frontend-Code und erzeugt UI-Komponenten auf Basis bestehender Figma-Designsysteme, ohne notwendigerweise Backend-Logik oder Geschäftsregeln zu erzeugen [5]. Weitere Werkzeuge wie Bolt.new

und v0 von Vercel besetzen ähnliche Nischen zwischen reinem UI-Prototyping und vollständiger Full-Stack-Generierung.

Tabelle 2.1 fasst die zentralen Vergleichsdimensionen dieser Werkzeuglandschaft zusammen. Trotz der Unterschiede im Detail zeigt sich ein gemeinsames Muster: Die für diese Arbeit relevanten Werkzeuge erzeugen häufig eine cloudzentrierte Web-Architektur mit einem JavaScript- oder TypeScript-Frontend und einem verwalteten Backend-Dienst. Der Begriff *cloudzentriert* bezeichnet dabei die in dieser Arbeit betrachtete Ausrichtung der dauerhaften Persistenz und zentralen Anwendungslogik auf entfernte Dienste. Er wird nicht mit dem weitergehenden Architekturbegriff *Cloud-Native* gleichgesetzt. Für das in dieser Arbeit betrachtete Transformationsszenario ist dies insofern relevant, als der Ausgangszustand des zu migrierenden Prototyps durch genau diese architektonischen Grundannahmen geprägt ist, unabhängig davon, mit welchem konkreten Werkzeug er erzeugt wurde.

Werkzeug	Zielsetzung	Bevorzugter Stack	Architektur-Charakter
Replit	Full-Stack-Prototyping	React/Node.js, PostgreSQL	Cloud-gebunden, prototyporientiert
Lovable	Full-Stack-Prototyping	React/Next.js, Supabase	Cloud-gebunden, BaaS-gekoppelt
Cursor / Copilot	Assistenzbasierte Codegenerierung	sprachagnostisch	In bestehende Codebasis eingebettet
Figma Make	Design-zu-Code	React/Vue/HTML+CSS	Statisch, reines Frontend
Bolt.new / v0	UI- und App-Prototyping	React/Next.js	Cloud-gebunden, deploy-mentororientiert

Tabelle 2.1: Vergleich KI-gestützter Entwicklungswerkzeuge nach Zielsetzung, Technologie-Stack und Architektur-Charakter

2.1.2 Charakteristika und Qualitätsdefizite KI-generierten Codes

KI-generierter Code entsteht als Ergebnis eines Sprachmodells, das plausible Fortsetzungen auf Basis von Trainingsdaten produziert, nicht jedoch auf Basis einer bewussten architektonischen Planung. Daraus resultiert ein Black-Box-Charakter: Designentscheidungen, Abhängigkeitsstrukturen und Sicherheitsannahmen werden vom Modell nicht dokumentiert und sind aus dem Code selbst häufig nicht unmittelbar rekonstruierbar. Dieser Befund lässt sich anhand des Produktqualitätsmodells nach ISO/IEC 25010 systematisch einordnen, das Softwarequalität in Merkmale wie Wartbarkeit, Sicherheit, Kompatibilität und Zuverlässigkeit unterteilt [6]. Empirische Untersuchungen zeigen für mehrere dieser Merkmale konsistente Defizite bei KI-generiertem Code.

Hinsichtlich der **Sicherheit** liefert eine wachsende Zahl unabhängiger Studien ein differenziertes, aber insgesamt problematisches Bild. Pearce *et al.* fanden bei einer systematischen Analyse von 1.689 durch GitHub Copilot generierten Programmen, dass rund 40 % der Vorschläge in sicherheitsrelevanten Szenarien verwundbaren Code enthielten [7]. Tihanyi *et al.* bestätigten ein hohes Fehlerrückkommen mit dem FormAI-v2-Datensatz, der durch neun verschiedene LLMs erzeugte C-Programme umfasst [8]. Asare *et al.* relativierten dieses Bild insofern, als Copilot bei der Reproduktion bekannter Schwachstellenmuster aus menschlich verursachten Common Weakness Enumerations (CWEs) tendenziell nicht pauschal schlechter abschnitt als die ursprünglichen menschlichen Entwickler [9]. Die Befunde sind daher nicht als Beleg einer generellen Unterlegenheit von KI-Code gegenüber menschlichem Code zu verstehen. Besonders relevant für die vorliegende Arbeit ist jedoch der Befund von Perry *et al.*, dass Entwicklerinnen und Entwickler mit Zugang zu einem KI-Assistenten nicht nur signifikant unsichereren Code produzierten, sondern zugleich überzeugter waren, sicheren Code geschrieben zu haben, als die Kontrollgruppe ohne KI-Unterstützung [10]. Dieses Auseinanderfallen von wahrgenommener und tatsächlicher Sicherheit begründet die Notwendigkeit einer unabhängigen Prüfung des Ausgangsartefakts.

Ein verwandtes, im Kontext von Lieferketten relevantes Risiko sind sogenannte *Package Hallucinations*. Spracklen *et al.* wiesen in einer Analyse von Codebeispielen aus verschiedenen LLMs nach, dass ein messbarer Anteil der von den Modellen vorgeschlagenen Paketabhängigkeiten nicht existierte [11]. Werden solche Namen ungeprüft in eine Paketkonfiguration übernommen, kann ein Angreifender ein Paket unter dem vorgeschlagenen Namen veröffentlichen und dadurch die Abhängigkeitsauflösung beeinflussen. Dieses Szenario ist kein notwendiges Merkmal jedes KI-generierten Projekts, stellt aber eine prüfbare Lieferkettengefahr dar.

Auch bezüglich der **Wartbarkeit** zeichnet sich ein negativer Trend ab. Eine Langzeitanalyse von GitClear über rund 153 Millionen geänderte Codezeilen zwischen 2020 und 2023 zeigt, dass der Anteil an kurzfristig wieder verworfenem oder überarbeitetem Code im Zuge der breiten Copilot-Adoption anstieg, während der Anteil an bewusst refaktoriertem Code zurückging [12]. Dieser Befund deckt sich mit der in der Praxis beobachtbaren Tendenz KI-generierten Codes zu geringer Kopplungsdisziplin und mangelnder Wiederverwendung bestehender Strukturen. Strukturell äußert sich dies häufig in einer unklaren Trennung von Präsentation, Geschäftslogik und Infrastruktur, da generierter Code primär auf die Erfüllung einer im Prompt beschriebenen Funktion optimiert wird.

Nicht zuletzt relativiert eine kontrollierte Studie der Forschungsorganisation METR das verbreitete Narrativ pauschaler Produktivitätssteigerungen durch KI-Assistenz. In einem randomisierten Experiment mit erfahrenen Open-Source-Entwicklerinnen und -Entwicklern, die reale Aufgaben in ihnen vertrauten Repositories bearbeiteten, benötigten die Teilnehmenden bei erlaubter KI-Nutzung im Mittel mehr Zeit als ohne KI-Unterstützung, obwohl sie im Vorfeld eine Beschleunigung erwartet hatten [13]. Dieser Befund ist nicht unmittelbar auf das No-Code-Prototyping durch technische Laien übertragbar, verdeutlicht aber, dass der Effekt von KI-Unterstützung auf Softwarequalität und Entwicklungsgeschwindigkeit kontextabhängig ist.

Die Konsequenzen dieser Qualitätsdefizite lassen sich anhand dokumentierter Vorfälle konkretisieren. Im Fall von Lovable wurde bekannt, dass ein Teil der untersuchten, mit dem Werkzeug erstellten Webanwendungen fehlerhafte Zugriffskontrollen in der angebundenen Supabase-Datenbank aufwies, wodurch sensible Daten öffentlich zugänglich waren [14]. In einem weiteren dokumentierten Fall löschte ein KI-Agent von Replit eine Produktionsdatenbank und meldete der nutzenden Partei anschließend fälschlicherweise, eine Wiederherstellung sei nicht möglich [15]. Beide Vorfälle illustrieren exemplarisch, dass die in empirischen Studien nachgewiesenen Qualitäts- und Sicherheitsdefizite nicht auf rein akademische Beispiele beschränkt sind.

Für die vorliegende Arbeit ergibt sich aus diesem Befund eine zentrale Implikation: KI-generierter Code kann nicht unverändert in eine produktionsreife, sicherheitskritische Desktop-Anwendung überführt werden. Die in den folgenden Abschnitten behandelten Konzepte zur Web-zu-Desktop-Transformation, zum Local-First-Paradigma und zur Absicherung lokaler Anwendungen bilden daher nicht nur eine technische, sondern auch eine aus den dargestellten Qualitätsrisiken begründete Notwendigkeit für ein systematisches Refactoring. Die methodische Herleitung dieses Refactorings erfolgt in Kapitel 3, während die konkrete Fallstudie in den Kapiteln 4 und 5 untersucht und umgesetzt wird.

2.2 Transformation von Web-Anwendungen in Desktop-Anwendungen

Die Transformation des untersuchten Prototyps erfordert zunächst eine begriffliche und technische Beschreibung des Ausgangszustands. Im Mittelpunkt steht eine cloudzentrierte Web-Referenzarchitektur, deren Benutzeroberfläche im Browser ausgeführt wird und deren dauerhaft maßgebliche Daten in einem entfernten Backend liegen. Anschließend wird Electron als Integrationsruntime eingeordnet und von alternativen Desktop-Runtimes abgegrenzt. Die dadurch entstehenden Vertrauensgrenzen und Sicherheitsmechanismen bilden die Grundlage für die Sicherheitsanforderungen in Abschnitt 2.3.

2.2.1 Referenzarchitektur KI-generierter Web-Anwendungen

Dieser Abschnitt beschreibt die in dieser Arbeit betrachtete Referenzarchitektur: einen KI-generierten, cloudzentrierten Web-Prototyp auf Basis eines clientseitigen Frontend-Frameworks und eines entfernten Backends. Die dargestellten Merkmale sind daher nicht als allgemeingültige Eigenschaften sämtlicher moderner Web-Anwendungen zu verstehen. Moderne Web-Anwendungen können ebenso serverseitig gerendert, offlinefähig, local-first oder mit lokaler Persistenz über Browsertechnologien umgesetzt werden. Im Mittelpunkt steht vielmehr der Ausgangszustand, der im weiteren Verlauf systematisch in eine lokal ausgeführte Desktop-Anwendung transformiert wird.

2.2.1.1 Request-Response-Modell und Zustandssemantik

Das Request-Response-Modell bildet eine wesentliche Grundlage vieler Web-Anwendungen. Eine nutzende Person löst über den Browser eine HTTP-Anfrage aus, ein Server verarbeitet diese, greift gegebenenfalls auf eine Datenbank zu und liefert eine Antwort zurück, beispielsweise in Form einer HTML-Seite oder von JSON-Daten [16]. Bei einer Single-Page Application (SPA) wird die Benutzeroberfläche nach dem initialen Laden typischerweise clientseitig aktualisiert. Weitere Daten werden asynchron über die Fetch API oder vergleichbare Schnittstellen abgerufen, ohne dass die gesamte Seite neu geladen werden muss [17].

HTTP stellt zwischen zwei Anfragen nicht automatisch einen Anwendungskontext bereit. Jeder Request muss daher grundsätzlich alle Informationen enthalten, die zu seiner Zuordnung und Verarbeitung erforderlich sind. Daraus folgt jedoch nicht, dass eine serverseitige Anwendung keinerlei Sitzungs-zustand verwalten kann. Ein Client kann beispielsweise ein Session-Cookie oder ein Token mitsenden, über das der Server auf einen zentral gespeicherten Sitzungs-zustand zugreift. Auf Anwendungsebene entsteht dadurch eine zustandsbehaftete Sitzung, obwohl HTTP selbst zwischen den Requests keinen Anwendungskontext fortschreibt [18].

Auch die Skalierbarkeit hängt nicht allein vom verwendeten Protokoll ab. Zustandslose Anwendungsinstanzen erleichtern die horizontale Skalierung, da Requests prinzipiell auf beliebige Instanzen verteilt werden können. Voraussetzung ist, dass sitzungsbezogener Zustand entweder im Request enthalten oder in einer gemeinsam erreichbaren Persistenzschicht abgelegt wird. Ein zentraler Session-Store vermeidet zwar eine Synchronisation zwischen den Anwendungsinstanzen, bleibt aber eine gemeinsam genutzte Abhängigkeit.

2.2.1.2 Browser als Präsentations- und Anwendungsschicht

In klassischen serverzentrierten Web-Architekturen wird der Browser überwiegend als Präsentationsschicht eingesetzt. Moderne Browser stellen jedoch verschiedene Mechanismen zur lokalen Persistenz bereit, darunter Cookies, Web Storage, IndexedDB und die Cache API. Ob und in welchem Umfang diese Mechanismen genutzt werden, hängt von der jeweiligen Anwendungsarchitektur ab [19].

SPAs haben die Aufgabenverteilung zwischen Client und Server zusätzlich verschoben. Frameworks wie React, Angular und Vue.js verlagern einen erheblichen Teil der Darstellungslogik in den Browser. Das Frontend verwaltet typischerweise UI-Zustand und kann darüber hinaus Teile der Validierungs-, Workflow- und Geschäftslogik ausführen. Hinzu kommen beispielsweise optimistische Aktualisierungen, clientseitige Berechnungen und lokale Caches.

Für die hier betrachtete Referenzarchitektur verbleibt die autoritative, dauerhaft persistierte Datenbasis jedoch im Cloud-Backend. Flüchtiger UI-Zustand oder ein clientseitiger Cache stellen dabei keine dauerhafte lokale Datenhaltung dar. Nach einem Seiten-Reload oder Neustart muss der für die Anwendung benötigte Datenbestand daher erneut aus dem Backend geladen werden, sofern keine zusätzliche Browserpersistenz implementiert wurde. Diese Abhängigkeit ist für die nachfolgende Transformation relevant, stellt aber keine allgemeine Eigenschaft von Web-Architekturen dar.

2.2.1.3 Cloud-Backend und autoritative Datenquelle

In der hier betrachteten cloudzentrierten Referenzarchitektur erfolgt die dauerhafte Datenhaltung überwiegend in einer entfernten Datenbank. Lese- und Schreiboperationen werden über Netzwerkschnittstellen ausgeführt, die beispielsweise nach einem REST- oder GraphQL-Muster strukturiert sein können. Die entfernte Datenbank dient dabei als autoritative Datenquelle für die dauerhaft gespeicherten Anwendungsdaten.

Im Kontext KI-generierter Web-Prototypen können Backend-as-a-Service-Angebote wie Supabase oder Firebase sowie integrierte Datenbankdienste von Entwicklungsplattformen zum Einsatz kommen. Diese Dienste reduzieren die initiale Entwicklungskomplexität, indem sie Datenbankbetrieb, Authentifizierung und teilweise auch clientseitige Zugriffsschnittstellen abstrahieren [4]. Das daraus entstehende Sicherheitsmodell beruht jedoch auf geteilter Verantwortung. Der Anbieter verantwortet unter anderem den Plattformbetrieb und technische Basismechanismen. Das Entwicklungsteam muss dagegen Datenmodell, Security Rules beziehungsweise Row-Level-Security-Regeln, Rollen, Zugriffskontrollen und die sichere Clientkonfiguration korrekt gestalten. Fehlkonfigurationen können dazu führen, dass Clients unzulässig auf Daten zugreifen [14].

Für die Referenzarchitektur sind vier Merkmale besonders relevant. Erstens liegt der dauerhaft gespeicherte Datenbestand überwiegend in einer entfernten Datenbank, während eine lokale Browserpersistenz nicht zwingend vorhanden ist. Zweitens sind zentrale Daten und Funktionen über Backend-Dienste erreichbar, sodass die davon abhängigen Funktionen ohne Netzwerkverbindung nicht oder nur eingeschränkt genutzt werden können. Drittens werden Datenänderungen über das Backend persistiert und anderen Clients je nach technischer Umsetzung bei einer erneuten Abfrage oder über Echtzeitmechanismen wie WebSockets beziehungsweise Subscriptions bereitgestellt. Viertens werden Benutzeridentitäten und Zugriffskontrollen typischerweise durch den Backend-Dienst verwaltet, während der Browser die zur Authentifizierung erforderlichen Informationen beispielsweise über Cookies oder Session-Tokens überträgt.

Der Begriff *Single Source of Truth* ist in diesem Zusammenhang nur präzise zu verwenden. Er kann sich auf die autoritative Persistenz, den UI-Zustand, einen Cache oder die Identitätsverwaltung beziehen. Im Folgenden ist mit der autoritativen Datenquelle ausschließlich der dauerhaft gespeicherte, fachlich maßgebliche Datenbestand gemeint.

2.2.1.4 Abhängigkeiten und Konsequenzen für die Transformation

Die beschriebene Web-Anwendung beruht auf einer Kette spezialisierter Abhängigkeiten. Diese Auflistung dient nicht als vollständige Beschreibung moderner Webtechnologien, sondern typisiert die für den betrachteten Ausgangszustand relevanten Schichten. HTTP beziehungsweise HTTPS und verbreitete Datenformate wie JSON bilden die Kommunikationsgrundlage. Frontend-Frameworks und UI-Bibliotheken wie React, Vue, Angular und Svelte unterstützen die Umsetzung der Benutzeroberfläche und werden durch Build- beziehungsweise Entwicklungswerkzeuge ergänzt. Client-State-Management verwaltet flüchtigen UI- und Anwendungszustand, während Bibliotheken wie TanStack Query entfernte Daten im Client puffern und aktualisieren können. Firebase, Supabase oder verwaltete Datenbanken stellen schließlich entfernte Persistenz- und Backendfunktionen bereit.

Die Unterscheidung zwischen Client-State und entfernten fachlichen Daten ist für die spätere Migration wesentlich. Ein lokaler UI-State beschreibt beispielsweise, ob ein Dialog geöffnet ist. Der fachlich maßgebliche Datenbestand liegt dagegen zunächst beim Backend. Eine lokale Desktop-Anwendung benötigt für diese Daten eine eigene dauerhafte Persistenz- und Zugriffsschicht, wenn sie unabhängig vom Netzwerk funktionieren soll.

Aus den beschriebenen Eigenschaften ergeben sich für die Transformation mehrere voneinander zu unterscheidende Aufgaben. Nicht das Request-Response-Modell selbst setzt eine permanente Netzwerkverbindung voraus, sondern die ausschließliche Abhängigkeit von entfernten Ressourcen. Sind Daten und zentrale Funktionen nur über Backend-Dienste verfügbar, können die betreffenden Funktionen ohne Netzwerkverbindung nicht oder nur eingeschränkt genutzt werden. Flüchtiger UI-State

und clientseitige Caches ersetzen keine eingebettete Persistenzschicht. Für die lokale Ausführung muss daher ein lokales Datenmodell, beispielsweise für SQLite, entwickelt und in die Anwendung integriert werden. Die direkte Kommunikation mit REST-, GraphQL- oder BaaS-Schnittstellen kann außerdem nicht ohne Weiteres durch eine lokale Datenbank ersetzt werden. Erforderlich sind Anpassungen der Datenzugriffsschicht, der Transaktionslogik und gegebenenfalls der Synchronisationsmechanismen. Schließlich ist zu entscheiden, ob die bestehende cloudbasierte Authentifizierung beibehalten, durch eine lokale Benutzerverwaltung ersetzt oder für einen Einzelbenutzerbetrieb vollständig entfernt wird.

Die lokale Ausführung einer Anwendung ist nicht automatisch mit einer Local-First-Architektur gleichzusetzen. Eine Desktop-Anwendung kann weiterhin ausschließlich auf ein Cloud-Backend zugreifen. Local First bezeichnet dagegen ein Datenhaltungs- und Synchronisationsmodell, bei dem lokale Datenbestände die primäre Grundlage der Interaktion bilden und Netzwerkzugriffe nicht für jede Operation erforderlich sind. Umgekehrt kann auch eine Browseranwendung local-first und offlinefähig umgesetzt werden.

Die hier untersuchte Transformation bezeichnet somit den Übergang von einer cloudzentrierten Webanwendung zu einer lokal ausgeführten Desktop-Anwendung mit dauerhafter lokaler Persistenz. Ob daraus zusätzlich ein Local-First-System entsteht, hängt von der konkreten Ausgestaltung der Datenhaltung und der Synchronisation ab. Unterabschnitt 2.2.2 stellt Electron als Brückentechnologie für die Ausführung von Webtechnologien auf dem Desktop vor. Unterabschnitt 2.2.3 diskutiert die daraus entstehenden sicherheitsrelevanten Konsequenzen. Kapitel 2.4 führt anschließend das Local-First-Paradigma als mögliches architektonisches Zielbild ein.

2.2.2 Desktop-Runtimes: Electron als Integrationsruntime für Web-Anwendungen

Die in dieser Arbeit betrachteten KI-generierten Web-Prototypen basieren auf einem JavaScript- beziehungsweise TypeScript-Stack und folgen einer webbasierten Client-Server-Architektur. Ihre Bereitstellung ist auf eine Ausführung im Browser und eine HTTP-basierte Kommunikation mit Backend-Diensten ausgerichtet. Für die lokale Ausführung wird daher eine Desktop-Runtime benötigt, die bestehende Webtechnologien in eine installierbare Anwendung einbettet und zugleich den kontrollierten Zugriff auf Betriebssystemressourcen ermöglicht.

Electron wird in dieser Arbeit als Integrationsruntime eingesetzt, weil es eine browserbasierte Benutzeroberfläche mit einer Node.js-basierten Laufzeitumgebung verbindet [20]. Die Bezeichnung Integrationsruntime beschreibt dabei die Funktion zwischen zwei Architekturebenen. Electron vermittelt zwischen der vorhandenen Web-Anwendung und den privilegierten Funktionen des Desktop-Betriebssystems. Dadurch können das bestehende Web-Frontend und Teile einer Node.js-basierten Anwendungs- oder Integrationslogik ohne grundlegenden Technologiebruch weiterverwendet werden. Die Migration konzentriert sich damit nicht auf eine vollständige Neuentwicklung der Benutzeroberfläche, sondern auf die kontrollierte Kapselung und Absicherung privilegierter Desktop-Funktionen.

2.2.2.1 Architektur und Prozessmodell

Im Zentrum der Electron-Anwendungsarchitektur stehen ein privilegierter Main-Prozess und ein oder mehrere Renderer-Prozesse. Electron verwendet darüber hinaus weitere interne Prozesse, beispielsweise für Grafik- und Hilfsfunktionen. Für die Sicherheits- und Integrationsbetrachtung dieser Arbeit ist jedoch insbesondere die Trennung zwischen Main- und Renderer-Prozess relevant [20].

Der Main-Prozess läuft in einer Node.js-Umgebung und verwaltet unter anderem Fenster, Betriebssystemintegration, Dateisystemzugriffe, native Module sowie die Kommunikation über Inter-Process Communication (IPC). Der Renderer-Prozess rendert die Benutzeroberfläche auf Chromium-Basis und führt den clientseitigen Web-Code aus. Privilegierte Funktionen sollten nicht unmittelbar im Renderer verfügbar sein, sondern über eine bewusst begrenzte Schnittstelle bereitgestellt werden.

Der Renderer-Prozess ist dabei nicht automatisch so sicher wie ein gewöhnlicher Browser-Tab. Seine tatsächliche Isolation hängt wesentlich von der Konfiguration der Electron-Anwendung und vom

eingebundenen Web-Code ab. Für eine geeignete Sicherheitsgrundlage müssen insbesondere die direkte Node.js-Integration im Renderer deaktiviert und die Context Isolation aktiviert werden. Zusätzlich kann Prozess-Sandboxing die Rechte des Renderer-Prozesses begrenzen. Ein Preload-Skript stellt kontrolliert ausgewählte Funktionen bereit, ersetzt aber weder die Context Isolation noch die Validierung der IPC-Kommunikation [21].

Die einzelnen Schutzmechanismen erfüllen unterschiedliche Aufgaben. Context Isolation trennt den JavaScript-Kontext des Web-Inhalts vom Kontext des Preload-Codes. Das Preload-Skript bildet eine kontrollierte Vermittlungsschicht zwischen Renderer und Main-Prozess. Sandboxing reduziert die verfügbaren Rechte des Renderer-Prozesses zusätzlich. Eine restriktive IPC-Schnittstelle und eine einseitige Validierung der übergebenen Daten begrenzen schließlich, welche privilegierten Operationen überhaupt ausgelöst werden können. Diese Maßnahmen bilden die technische Grundlage für die im folgenden Unterabschnitt untersuchte Härtung der Anwendung [21, 22].

2.2.2.2 Betrachtete Alternativen

Die Wahl von Electron ergibt sich nicht daraus, dass es die einzige verfügbare Desktop-Runtime wäre. Für die vorliegende Transformation sind vielmehr die Wiederverwendbarkeit des bestehenden Codes, die Kompatibilität mit Node.js-basierter Integrationslogik, die verfügbaren Sicherheitsmechanismen, der Zugriff auf Betriebssystemressourcen sowie der erwartete Migrations- und Betriebsaufwand maßgeblich.

Tauri verbindet ebenfalls ein Web-Frontend mit einer nativen Desktop-Shell und verwendet grundsätzlich die WebView-Komponente des jeweiligen Betriebssystems anstelle eines gebündelten Chromium-Browsers [23]. Das kann zu kleineren Binärgrößen und einem geringeren Ressourcenverbrauch führen. Das native Backend ist jedoch primär Rust-basiert. Zwar lassen sich externe Prozesse oder sogenannte Sidecars integrieren, eine unmittelbare und gleichwertige Weiterverwendung einer Node.js-basierten Backend-Logik entspricht jedoch nicht dem Standardmodell von Tauri. Für das betrachtete Szenario würde dies zusätzlichen Integrationsaufwand oder eine partielle Neuimplementierung privilegierter Anwendungslogik erfordern.

Ein vereinfachtes Zielbild stellt das reine Verpacken einer bestehenden Webseite als Desktop-Anwendung dar. Das Projekt Pake nutzt Tauri, um Webseiten oder lokale Web-Builds mit geringem Konfigurationsaufwand für mehrere Betriebssysteme zu paketieren [24]. Dieses Vorgehen reduziert den Implementierungs- und Paketierungsaufwand, verändert jedoch weder die zugrunde liegende Datenhaltung noch die Abhängigkeit von entfernten Backend-Diensten. Es eignet sich daher insbesondere für Anwendungen, bei denen ein Desktop-Fenster und eine einfache Plattformintegration ausreichen. Für das vorliegende Transformationsszenario ist ein solcher Wrapper nicht ausreichend, da lokale Persistenz, Offline-Fähigkeit, Datensouveränität und die kontrollierte Ablösung des Cloud-Backends zentrale Anforderungen darstellen.

NW.js verfolgt ebenfalls einen Chromium- und Node.js-basierten Ansatz und weist dadurch eine hohe technische Nähe zu Electron auf [25]. Für die Auswahlentscheidung ist daher nicht das Fehlen eines einzelnen Features ausschlaggebend. Maßgeblich ist vielmehr, dass Electron für das in dieser Arbeit verfolgte Ziel eine gut dokumentierte Grundlage für die systematische Konfiguration von Prozessisolation, Preload-Schnittstellen und Content Security Policy bietet.

Flutter Desktop stellt eine weitere Möglichkeit zur Entwicklung plattformübergreifender Desktop-Anwendungen dar, verwendet jedoch mit Dart und einer eigenen Rendering-Architektur ein grundsätzlich anderes Sprach-, Rendering- und Programmiermodell [26]. Die vorhandene HTML-, CSS-, JavaScript- und TypeScript-basierte Benutzeroberfläche könnte daher nicht unmittelbar übernommen werden. Eine Entscheidung für Flutter würde neben dem Build-Prozess auch das UI- und Anwendungsmodell grundlegend verändern und wäre für das Wiederverwendungsziel dieser Arbeit mit einem hohen Neuentwicklungsaufwand verbunden.

2.2.2.3 Begründung der Auswahl

Unter den betrachteten Desktop-Runtimes weist Electron die höchste Kompatibilität mit dem JavaScript- und TypeScript-Stack der Ausgangsanwendung auf. Die Web-UI kann in der vorhandenen Technologie weitergeführt werden, während Node.js-basierte Komponenten für die lokale Integration und den Zugriff auf Betriebssystemressourcen grundsätzlich anschlussfähig bleiben. Tauri bietet zwar Vorteile bei Binärgröße und Ressourcennutzung, erfordert für native Anwendungslogik jedoch ein anderes primäres Programmiermodell. NW.js ist technisch vergleichbar, stellt für das hier untersuchte Sicherheitsziel aber die weniger geeignete Referenz für eine systematische Härtung dar. Flutter Desktop würde dagegen eine weitgehende Neuentwicklung der vorhandenen Web-Oberfläche voraussetzen.

Electron wird deshalb als Zielruntime ausgewählt, weil die Technologie eine weitgehende Wiederverwendung des bestehenden Web- und Node.js-Stacks mit einer kontrollierten Kapselung privilegierter Desktop-Funktionen verbindet. Diese Wiederverwendbarkeit reduziert den Migrationsaufwand, ersetzt jedoch keine Sicherheitsanalyse. Insbesondere die Trennung von Main- und Renderer-Prozessen, die Gestaltung der Preload-API, die Absicherung der IPC-Kommunikation, die Begrenzung von Dateisystemzugriffen und der Umgang mit nicht vertrauenswürdigen Web-Inhalt sind für die Sicherheit der resultierenden Desktop-Anwendung maßgeblich.

2.2.3 Sicherheitsmodell von Electron

Electron ist keine automatisch sichere Ausführungsumgebung. Die Runtime stellt jedoch Prozess- und Kontextgrenzen bereit, mit denen privilegierte Desktop-Funktionen von nicht privilegiertem Web-Code getrennt werden können. Die Wirksamkeit dieser Grenzen hängt von der Konfiguration der Anwendung, der Gestaltung der Preload- und IPC-Schnittstellen sowie von der Vertrauenswürdigkeit der geladenen Inhalte ab [21]. Dieser Abschnitt beschreibt die technischen Grundlagen dieses Sicherheitsmodells. Die Bedrohungen für lokale Desktop-Anwendungen, die spezifischen Risiken KI-generierten Codes und die daraus abgeleiteten Anforderungen werden anschließend in Abschnitt 2.3 behandelt.

2.2.3.1 Prozesse und Vertrauensgrenzen

Für die Sicherheitsbetrachtung sind drei Begriffe getrennt zu halten. Die Vertrauenszone beschreibt, welchem Code vertraut wird. Privilegien beschreiben die verfügbaren Fähigkeiten eines Prozesses oder einer Schnittstelle. Betriebssystemrechte begrenzen schließlich den tatsächlichen Zugriff auf Systemressourcen. Diese Ebenen dürfen nicht synonym verwendet werden. Eine privilegierte Position im Prozessmodell bedeutet nicht automatisch eine uneingeschränkte Freigabe auf Betriebssystemebene.

Im Prozessmodell von Electron ist zwischen dem Renderer-Prozess, dem Preload-Skript und dem Main-Prozess zu unterscheiden [20]. Der Renderer-Prozess verarbeitet HTML, CSS und JavaScript und ist grundsätzlich als nicht vertrauenswürdiger beziehungsweise weniger privilegierter Web-Code zu behandeln. Das Preload-Skript wird vor dem Laden der Web-Anwendung ausgeführt und stellt privilegierten Vermittlungscode mit einer begrenzten, veröffentlichten Capability-Schnittstelle dar. Der Main-Prozess läuft in einer Node.js-Umgebung und bildet privilegierte Anwendungslogik mit potenziell weitreichenden Node.js- und Betriebssystemzugriffen auf Dateisystem, Prozesse oder native Module. Der Main-Prozess besitzt damit nicht automatisch nur die für die Anwendung erforderlichen Rechte.

Das Preload-Skript kann ausgewählte Funktionen für den Renderer verfügbar machen, ohne dem Web-Inhalt automatisch alle Node.js- oder Electron-APIs zu überlassen. Es gehört dennoch selbst zur sicherheitskritischen Vertrauensgrenze. Eine zu weitreichende Schnittstelle kann die beabsichtigte Isolation trotz einer ansonsten restriktiven Prozesskonfiguration schwächen. Electron verwendet intern neben Main- und Renderer-Prozessen weitere Prozesse, beispielsweise für Grafik-, Hilfs- und Absturzbehandlungsfunktionen. Für die vorliegende Arbeit ist jedoch vor allem die Unterscheidung zwischen privilegiertem Anwendungscode und nicht privilegiertem Web-Inhalt relevant.

2.2.3.2 Isolation des Renderer-Prozesses

Die wichtigste Sicherheitsentscheidung besteht darin, dem Renderer keinen direkten Zugriff auf privilegierte APIs zu geben. Die Node.js-Integration sollte im Renderer deaktiviert werden, sodass dort ausgeführter Web-Code nicht unmittelbar auf `require`, das Dateisystem oder andere Node.js-Funktionen zugreifen kann [21].

Zusätzlich sollte die Context Isolation aktiviert werden. Sie trennt die direkten JavaScript-Kontexte des geladenen Web-Inhalts und des Preload-Codes und verhindert, dass Web-Code globale Objekte des Preload-Kontexts unmittelbar verändert. Über eine Bridge veröffentlichte Funktionen bleiben jedoch absichtlich erreichbar. Context Isolation ersetzt diese Funktionen nicht und beschränkt sie nicht automatisch [27].

Das Prozess-Sandboxing kann die verfügbaren System- und Laufzeitfunktionen des Renderer-Prozesses weiter einschränken. Es ist nicht mit vollständiger Betriebssystemisolation gleichzusetzen, und seine konkrete Wirkung hängt von der verwendeten Electron-Version, der Fensterkonfiguration und der Preload-Ausführung ab. Das Prozess-Sandboxing betrifft den Renderer-Prozess. Die privilegierte Anwendungslogik im Main-Prozess wird dadurch nicht in gleicher Weise eingeschränkt [28].

Die Isolation des Renderers ist keine vollständige Sicherheitsgarantie. Sie begrenzt zunächst nur den direkten Zugriff. Ein kompromittierter Renderer kann weiterhin versuchen, erlaubte Preload- oder IPC-Funktionen missbräuchlich aufzurufen. Die Sicherheit der Anwendung hängt daher ebenso von der Begrenzung dieser Schnittstellen ab.

2.2.3.3 Preload als Capability-Schnittstelle

Das Preload-Skript sollte ausschließlich die Funktionen bereitstellen, die der Renderer für die fachliche Interaktion benötigt. Statt allgemeine Systemoperationen zu veröffentlichen, sollte die Schnittstelle fachliche Fähigkeiten anbieten. Eine Capability bezeichnet dabei eine nicht beliebig übertragbare Berechtigung zur Ausführung einer konkreten Operation. Eine Funktion wie `saveStudyPlan(plan)` begrenzt den möglichen Zugriff stärker als eine allgemeine Funktion wie `writeFile(path, data)`, die beliebige Pfade und Inhalte annehmen könnte.

Die Veröffentlichung solcher Funktionen erfolgt typischerweise über eine kontrollierte Bridge, beispielsweise `contextBridge`. Das vollständige Electron- oder Node.js-API darf nicht an den Renderer weitergereicht werden. Ebenso sollten keine rohen IPC-Funktionen wie `ipcRenderer.send` oder `ipcRenderer.invoke` öffentlich gemacht werden. Andernfalls könnte der Renderer beliebige Kanäle auswählen und die vorgesehene fachliche Begrenzung umgehen [27, 22].

Auch Rückgabewerte und Fehlermeldungen gehören zur Schnittstellengestaltung. Sie sollten auf die fachliche Domäne der Anwendung begrenzt werden und keine internen Dateipfade, Stacktraces oder sonstigen sensiblen Informationen offenlegen. Die Preload-API ist damit als schmaler und überprüfbarer Vertrag zwischen Benutzeroberfläche und privilegierter Anwendungslogik zu verstehen.

2.2.3.4 IPC als Kommunikationsgrenze

IPC verbindet den Renderer mit dem Main-Prozess und ist damit ein zentraler Bestandteil der Electron-Anwendungsarchitektur [22]. Jeder IPC-Kanal stellt eine kontrollierte Verbindung in eine privilegierte Vertrauenszone dar. Die Tatsache, dass eine Anfrage aus dem eigenen Renderer stammt, reicht deshalb nicht aus, um ihre Eingaben als vertrauenswürdig anzunehmen.

Neben der Validierung der Eingabedaten ist der Absender jeder IPC-Nachricht zu prüfen. IPC-Nachrichten können von unerwarteten Frames oder eingebetteten Inhalten ausgelöst werden. Der Main-Prozess sollte den aufrufenden Renderer beziehungsweise Frame anhand einer Origin- oder Fenster-Allowlist validieren, bevor eine Anfrage weiterverarbeitet wird [21].

Die Kanäle sollten eine eindeutige fachliche Verantwortung besitzen. Das Preload-Skript sollte nicht als allgemeiner Transport für beliebige Renderer-Anfragen dienen, sondern nur klar definierte Operationen

vermitteln. Die konkrete Validierung von Datentypen, Wertebereichen, Ressourcenbezügen, Absendern und Dateipfaden wird im Main-Prozess beziehungsweise in der privilegierten Anwendungsschicht durchgeführt. Die Pfadprüfung reduziert dabei das Risiko von Pfadüberquerungen (Path Traversal), schließt aber weitere Dateisystemrisiken wie symbolische Links oder konkurrierende Änderungen nicht automatisch aus.

IPC ist damit nicht lediglich ein technisches Transportmittel. Es bildet einen API-Vertrag zwischen zwei unterschiedlich privilegierten Ausführungskontexten. Seine konkrete Absicherung und die daraus folgenden Prüfanforderungen werden in Abschnitt 2.3 als Teil des Bedrohungsmodells und der Sicherheitsanforderungen wieder aufgegriffen.

2.2.3.5 Geladene Inhalte und zusätzliche Schutzschichten

Die Vertrauenswürdigkeit der geladenen Inhalte ist ein weiterer Bestandteil des Electron-Sicherheitsmodells. Eine Anwendung sollte möglichst nur erwartete lokale Inhalte oder ausdrücklich erlaubte Ursprünge laden. Für das konkrete Szenario ist daher festzulegen, welche URL-Schemata erlaubt sind, ob ausschließlich gebündelte lokale Inhalte geladen werden, wie Navigation und neue Fenster behandelt werden und ob externe Links unterstützt werden. Externe Webseiten, unkontrollierte Weiterleitungen und unnötige Remote-Ressourcen vergrößern die Vertrauensbasis der Anwendung [21].

Eine Content Security Policy (CSP) kann die Ausführung unerwarteter Skripte und die Einbindung nicht erlaubter Ressourcen begrenzen. Ihre konkrete Wirksamkeit hängt unter anderem davon ab, ob sie über einen HTTP-Header oder ein Meta-Tag ausgeliefert wird und über welches Protokoll die Inhalte geladen werden. Bei lokal über `file://` geladenen Inhalten sind deshalb Ladeprotokoll und Build- beziehungsweise Deployment-Strategie zu berücksichtigen [21]. CSP bildet eine zusätzliche Verteidigungsschicht gegen bestimmte Angriffe auf den Renderer, ersetzt jedoch weder eine sichere Datenbehandlung noch die Validierung privilegierter Anfragen. Die Deaktivierung von Web-Sicherheitsmechanismen aus Gründen der Entwicklungsbequemlichkeit sollte nicht Bestandteil einer Produktionskonfiguration sein.

2.2.3.6 Bedeutung für die Transformation

Für die Transformation einer Web-Anwendung in eine Electron-Anwendung folgt daraus, dass der vorhandene Web-Code nicht unverändert in einen privilegierten Prozess verschoben werden darf. Der Renderer übernimmt Darstellung und nicht privilegierte Interaktion. Das Preload-Skript stellt minimale, fachlich definierte Fähigkeiten bereit. Der Main-Prozess führt privilegierte Operationen über kontrollierte Schnittstellen aus. Dabei verlangt das Prinzip der geringsten Rechte, dass sowohl die verfügbaren Fähigkeiten als auch die Betriebssystemrechte auf das für die Funktion erforderliche Maß begrenzt werden.

Eine Kompromittierung des Renderers ist nicht als zwangsläufige Angriffskette, sondern als möglicher Angriffspfad zu verstehen. Je nach Origin, Frame, Preload-API, IPC-Senderprüfung und Betriebssystemrechten ergeben sich unterschiedliche Auswirkungen. Diese reichen von Cross-Site-Scripting (XSS) im Renderer über eine Privilege Escalation über die Preload- oder IPC-Schnittstelle bis hin zu einer Remote Code Execution (RCE) im Main-Prozess.

Schließlich sind Laufzeit- und Lieferkettensicherheit getrennt zu betrachten. Electron, Chromium, Node.js, native Module und über den Paketmanager bezogene Abhängigkeiten sind Laufzeitbestandteile der ausgelieferten Anwendung. Ihre Versionen, Herkunft und Aktualisierungsverfahren müssen kontrolliert werden [21]. Build-Werkzeuge und der Buildprozess gehören dagegen nicht in gleicher Weise zur Laufzeitumgebung, sind aber für die Integrität des erzeugten Artefakts relevant.

Electron bietet somit eine technische Grundlage für die Trennung von Web-Inhalt und Desktop-Funktionalität, erzwingt diese Trennung aber nicht. Die sicherheitsrelevante Transformationsleistung besteht darin, die Vertrauensgrenze explizit zu entwerfen und die Übergänge zwischen den Ausführungskontexten kontrollierbar zu machen. Welche Bedrohungen daraus für das konkrete Artefakt entstehen und welche Anforderungen sich ableiten, wird im folgenden Abschnitt untersucht.

2.3 Sicherheit lokaler Desktop-Anwendungen

Abschnitt 2.2.3 beschreibt das technische Sicherheitsmodell von Electron. Der vorliegende Abschnitt nimmt darauf auf und betrachtet die Sicherheitsprobleme aus Sicht der lokalen Desktop-Zielanwendung beziehungsweise des konkreten Transformationsszenarios des Studienplaners. Im Mittelpunkt stehen die zu schützenden Ressourcen, mögliche Angriffsflächen und die besonderen Risiken eines KI-generierten Ausgangssystems. Die Electron-Mechanismen werden dabei nicht erneut als technische Dokumentation vorgestellt. Sie dienen vielmehr als Grundlage für die Ableitung von Sicherheitszielen und überprüfbaren Anforderungen an das Zielartefakt.

2.3.1 Bedrohungsmodell und Angriffsflächen

Die hier betrachtete lokale Desktop-Zielanwendung besitzt andere Vertrauensannahmen als die cloud-zentrierte Web-Referenzarchitektur des Ausgangsartefakts. In der Web-Referenzarchitektur liegt die Durchsetzung von Zugriffsgrenzen maßgeblich im Backend. In der Zielanwendung befinden sich Daten, Programmcode und Laufzeit zumindest teilweise auf dem Endgerät. Dadurch werden serverseitige Schutzgrenzen durch lokale Kontroll- und Vertrauensgrenzen ersetzt, während neue Angriffsflächen entstehen.

Threat Modeling bezeichnet eine strukturierte Betrachtung von Angriffs- und Verteidigungsaspekten eines Schutzobjekts, einer Anwendung oder eines Systems [29]. Für das vorliegende Szenario werden die Schutzziele Vertraulichkeit, Integrität und Verfügbarkeit als Ordnungsrahmen verwendet. Die Betrachtung ist auf die Electron-Zielanwendung und die lokale Datenhaltung des Studienplaners begrenzt. Sie beansprucht daher nicht, sämtliche Bedrohungen allgemeiner Desktop- oder Web-Anwendungen abzudecken.

Zu den relevanten Schutzobjekten gehören im Studienplaner-Szenario insbesondere die lokalen Studienplandaten mit Kursen, Prüfungen und Semesterdaten, die SQLite-Datenbank, Konfigurationsdateien sowie die Integrität der Anwendung selbst. Übernommene Zugangsdaten oder Token des cloud-zentrierten Ausgangssystems sind nur während einer Übergangs- oder Migrationsphase als zusätzliche Schutzobjekte relevant. Diese Schutzobjekte sind für den Studienplaner bedeutsam, weil die Studienplandaten das zentrale Arbeitsergebnis darstellen und ihr Verlust oder ihre unbefugte Veränderung die fachliche Nutzung unmittelbar beeinträchtigen würde.

Als mögliche Angriffsflächen sind der Renderer-Prozess, die Preload- und IPC-Schnittstellen, Dateisystemzugriffe, externe Links, Netzwerkfunktionen, native Module sowie Paketierungs- und Updateprozesse zu berücksichtigen. Ein mögliches Angriffsszenario beginnt mit manipulierten oder unerwarteten Eingaben im Renderer. Werden diese Eingaben über eine zu weitreichende Preload- oder IPC-Schnittstelle an den Main-Prozess weitergegeben, können daraus unzulässige Dateisystem- oder Prozessoperationen entstehen. Die konkrete Auswirkung hängt dabei nicht nur vom ursprünglichen Fehler, sondern von den erreichbaren Fähigkeiten und den Rechten der privilegierten Anwendungsschicht ab.

Die Schutzziele lassen sich auf die Angriffsflächen beziehen. Die Vertraulichkeit betrifft beispielsweise Studienplandaten, Konfigurationen und mögliche Zugangsdaten. Die Integrität betrifft Datenbankinhalte, Migrationen, Programmdateien und die Semantik fachlicher Operationen. Die Verfügbarkeit betrifft den Zugriff auf die lokale Datenbank, den erfolgreichen Start der Anwendung und die Wiederherstellung nach Fehlern. Diese Zuordnung unterstützt die spätere Analyse, ersetzt aber keine konkrete Prüfung des Ausgangs- und Zielartefakts.

2.3.2 Sicherheitsrisiken KI-generierter Anwendungen

Die in Abschnitt 2.1.2 dargestellten Untersuchungen zeigen, dass KI-gestützt generierter Code in relevanten Szenarien sicherheitskritische Fehler enthalten kann [10, 7, 8, 9]. Ob und in welchem Umfang diese Befunde auf den Studienplaner zutreffen, ist eine Arbeitsannahme der Fallstudie und wird in der späteren Analyse anhand des konkreten Ausgangsartefakts geprüft.

Besonders relevant für die Transformation sind fehlende Eingabevalidierung, überprivilegierte Hilfsfunktionen, unsichere Standardkonfigurationen und eine unklare Trennung von Präsentation, Geschäftslogik und Infrastruktur. Weitere mögliche Risiken betreffen hartkodierte Geheimnisse, unzureichende Fehlerbehandlung, fehlende Zugriffskontrollen und unsichere Abhängigkeiten. Diese Kategorien sind nicht als bereits nachgewiesene Eigenschaften des Studienplaners zu verstehen. Sie bilden Prüfkategorien für die Analyse des generierten Codes und lassen sich mit etablierten Kategorien wie Injection, Broken Access Control und Security Misconfiguration in Beziehung setzen [30].

Darüber hinaus kann KI-generierter Code die Lieferkette gefährden. Wenn ein Modell einen derzeit nicht existierenden Paketnamen vorschlägt, kann unter diesem Namen ein böses Paket veröffentlicht und im Rahmen der Abhängigkeitsauflösung bezogen werden [11]. Die Gefahr entsteht dabei nicht durch die sprachliche Generierung allein, sondern durch die ungeprüfte Übernahme des Vorschlags in eine ausführbare Paketkonfiguration. Ein reproduzierbarer Installationsprozess, ein versioniertes Lockfile und die Prüfung der Paketquelle sind deshalb Bestandteile der späteren Härtung.

Die Sicherheitsrisiken des KI-generierten Ausgangssystems verbinden sich mit den Risiken der lokalen Desktop-Runtime. Eine fehlende Validierung kann im Web-Prototyp zunächst eine unzulässige Serveranfrage auslösen. Nach der Transformation kann dieselbe unklare Eingabegrenze jedoch eine privilegierte lokale Operation erreichen. Ein unkritisch übernommener Dateizugriff, ein zu allgemeiner IPC-Kanal oder eine unsichere Navigation erhalten dadurch eine andere Schadenswirkung. Die Analyse muss daher sowohl die Qualität des generierten Codes als auch seine Position innerhalb der neuen Vertrauensgrenzen bewerten.

2.3.3 Sicherheitsanforderungen an das Zielartefakt

Aus dem Bedrohungsmodell in Abschnitt 2.3.1 und den Risiken des KI-generierten Ausgangscodes werden Zielerfordernisse abgeleitet. Sie sind bewusst als zu erfüllende Anforderungen und nicht als bereits gesicherte Eigenschaften formuliert.

Der Renderer darf nicht direkt auf das Dateisystem, native Module oder allgemeine Node.js-APIs zugreifen. Privilegierte Operationen müssen über eine minimale und fachlich definierte Preload- beziehungsweise IPC-Schnittstelle erfolgen. Eingaben aus dem Renderer müssen vor jeder privilegierten Operation erneut validiert werden. Zusätzlich muss der Main-Prozess den aufrufenden Renderer beziehungsweise Frame prüfen, damit nicht erwartete oder eingebettete Inhalte die Schnittstelle missbrauchen können.

Dateipfade müssen auf erlaubte Verzeichnisse, Dateitypen und Operationen begrenzt werden. Die Pfadvalidierung soll insbesondere Pfadüberquerungen verhindern, darf aber nicht als vollständiger Schutz vor allen Dateisystemrisiken verstanden werden. Sensible Daten dürfen nicht in Quellcode, Renderer-Bundles oder Protokollmeldungen eingebettet werden. Die lokale Datenbank ist in einem beschreibbaren Benutzerdatenpfad getrennt vom unveränderlichen Installationsverzeichnis zu verwalten. Für sensible Datenbestände ist zusätzlich zu prüfen, ob Betriebssystemrechte ausreichen oder eine Verschlüsselung ruhender Daten und eine geeignete Schlüsselverwaltung erforderlich sind.

Abhängigkeiten müssen nachvollziehbar, reproduzierbar und auf bekannte Quellen zurückführbar sein. Dazu gehören insbesondere eine versionierte Paketauflösung, eine Prüfung der Paketnamen und ein kontrollierter Umgang mit nativen Modulen. Fehlerzustände müssen kontrolliert behandelt werden und dürfen keine internen Pfade, Stacktraces oder sonstigen sensiblen Informationen offenlegen. Der Update- und Paketierungsprozess muss gegen manipulierte oder gefälschte Installationen abgesichert sein.

Die Sicherheitsanforderungen müssen durch statische Prüfungen, Konfigurationskontrollen oder Tests überprüfbar sein. Die konkrete Umsetzung der IPC-Kanäle, Validatoren, Datenzugriffsfunktionen und Build-Konfigurationen wird deshalb nicht in diesem Grundlagenabschnitt vorweggenommen, sondern in den folgenden Kapiteln anhand der Fallstudie dokumentiert. Die Anforderungen bilden die Verbindung zwischen dem theoretischen Sicherheitsmodell, der Analyse des Ausgangsartefakts, dem Entwurf der Zielarchitektur und der späteren Evaluation.

2.4 Das Local-First-Paradigma

Local First bezeichnet keine einzelne Technologie und keine bestimmte Desktop-Runtime, sondern ein Architekturleitbild. Daten und zentrale Anwendungsoperationen sollen zunächst auf dem Gerät der nutzenden Person stattfinden, während ein Netzwerkdienst lediglich ergänzend für Synchronisation, Sicherung oder gemeinsame Nutzung eingesetzt wird. Für die vorliegende Arbeit wird diese Beschreibung als Arbeitsdefinition verwendet und an die konzeptionelle Darstellung von Kleppmann *et al.* angelehnt [31]. Damit unterscheidet sich Local First von der im Ausgangsartefakt betrachteten serverzentrierten Architektur, in der die entfernte Datenbank die autoritative Datenquelle darstellt und der Client überwiegend als Repräsentation oder Cache dieser Daten dient.

Die lokale Ausführung ist eine mögliche, aber keine hinreichende Voraussetzung für Local First. Eine Electron-Anwendung kann weiterhin für jede fachliche Operation ein entferntes Backend benötigen. Umgekehrt kann eine Webanwendung local-first-Eigenschaften besitzen, ohne als Desktop-Anwendung installiert zu sein. Für die vorliegende Arbeit sind daher die Desktop-Runtime, die lokale Persistenz und das Local-First-Paradigma begrifflich zu trennen.

2.4.1 Kernprinzipien nach Kleppmann et al.

Auf der Literaturebene entwerfen Kleppmann *et al.* Local First als Gegenentwurf zu einem ausschließlich serverzentrierten Datenmodell [31]. Die lokale Kopie der Daten gilt als primäre Kopie. Ein Server kann sekundäre Kopien für Synchronisation, Datensicherung oder den Zugriff weiterer Geräte bereitstellen. Die Existenz eines Servers widerspricht dem Paradigma daher nicht, solange die lokale Anwendung nicht von dessen ständiger Verfügbarkeit abhängt und die lokale Kopie nicht als entbehrlicher Cache behandelt wird.

Die von Kleppmann *et al.* formulierten sieben Ideale dienen in dieser Arbeit als konzeptionelle Orientierung, nicht als technische Checkliste, die durch den Einsatz einer bestimmten Datenbank automatisch erfüllt würde. Das erste Ideal, *No spinners*, beschreibt die unmittelbare Verfügbarkeit der Anwendung. Lese- und Schreiboperationen sollen nicht von der Antwortzeit eines entfernten Servers abhängen, sondern zunächst lokal verarbeitet und anschließend gegebenenfalls synchronisiert werden. Lokale Datenhaltung garantiert dabei keine beliebig geringe Latenz, schafft aber die architektonische Voraussetzung für eine vom Netzwerk unabhängige Interaktion.

Das zweite Ideal betrifft die Verfügbarkeit auf mehreren Geräten. Die lokale Primärkopie soll auf weiteren Geräten verfügbar sein, ohne die lokale Autonomie aufzugeben. Dazu müssen Änderungen übertragen und gegebenenfalls zusammengeführt werden. Synchronisation ist daher kein bloßer Kopiervorgang, sondern muss parallele Änderungen mehrerer Geräte berücksichtigen.

Das dritte Ideal beschreibt das Netzwerk als optionale Infrastruktur. Die nutzende Person soll Daten auch ohne Verbindung lesen und verändern können, während ausstehende Änderungen nach Wiederherstellung der Verbindung synchronisiert werden. Dabei ist zwischen einer kurzen Unterbrechung und einem vollständig offlinefähigen System zu unterscheiden. Offlinefähigkeit setzt ein lokales Datenmodell, lokale Geschäftsoperationen und eine Strategie für ausstehende Änderungen voraus.

Das vierte Ideal ist die nahtlose Zusammenarbeit. Parallele Änderungen sollen erkannt und möglichst automatisch zusammengeführt werden. Bei fachlich strukturierten, semantisch abhängigen Daten ist eine rein textbasierte Zusammenführung jedoch häufig unzureichend, weil die fachliche Bedeutung nicht aus der Zeichenfolge hervorgeht. Kollaboration ist deshalb eine zusätzliche Komplexitätsstufe und keine zwingende Voraussetzung jeder Local-First-Anwendung.

Das fünfte Ideal, *The Long Now*, richtet den Blick auf die langfristige Erhaltung der Daten. Daten sollen auch bei Wegfall eines Anbieters zugänglich bleiben. Die daraus für diese Arbeit abgeleitete technische Konsequenz besteht darin, Daten in einem nachvollziehbaren Format zu speichern, zu exportieren und gegebenenfalls in zukünftige Schema-Versionen überführen zu können. Die konkrete Forderung nach versionierten Schema-Migrationen ist dabei eine technische Ausarbeitung der vorliegenden Arbeit und keine unmittelbare Eigenschaft der Local-First-Quelle.

Das sechste Ideal behandelt Sicherheit und Datenschutz als Standard. Eine zentrale Datenbank mit den Daten sämtlicher Nutzender ist für eine local-first-Anwendung nicht zwingend erforderlich. Optionale Synchronisation kann durch Ende-zu-Ende-Verschlüsselung die Einsichtsmöglichkeiten eines Servers begrenzen. Local First ist jedoch nicht automatisch sicherer. Das Paradigma kann die zentrale Datensammlung reduzieren, ersetzt aber weder den Schutz des Endgeräts noch die Sicherheit der konkreten Anwendung. Lokale Daten müssen deshalb gegen unberechtigten Zugriff, Schadsoftware, Verlust und unsichere Backups geschützt werden.

Das siebte Ideal betrifft die Kontrolle und den Besitz der Daten. Gemeint ist damit im Rahmen dieser Arbeit nicht juristisches Eigentum, sondern die Möglichkeit, Daten unabhängig von den Vorgaben eines Dienstansbieters zu lesen, zu kopieren, zu exportieren, zu sichern und weiterzuverarbeiten. Ein eingeschränktes Exportformat oder eine nur über proprietäre APIs erreichbare Datenbank begrenzt diese Kontrolle trotz lokaler Zwischenspeicherung.

Die sieben Ideale gehen damit über die Forderung “offline verfügbar” hinaus. Offlinefähigkeit, geringe Latenz und lokale Persistenz bilden die technische Grundlage, während Datensouveränität, langfristige Erhaltung, Datenschutz und gegebenenfalls Kollaboration die Nutzung und den Lebenszyklus der Daten betreffen. Nicht jede Anwendung muss alle Ideale im gleichen Umfang erfüllen. Ein lokales Einzelplatzszenario kann von lokaler Verfügbarkeit, geringer Latenz, Datenkontrolle und langfristig lesbaren Daten profitieren, ohne Echtzeitkollaboration anzubieten.

Für die Zusammenführung paralleler Änderungen sind auf der Literaturebene insbesondere Operational Transformation und Conflict-free Replicated Data Types (CRDTs) relevant. CRDTs verwenden Datenstrukturen, deren Zusammenführung unter bestimmten Voraussetzungen reihenfolgeunabhängig zu einem konvergenten Zustand führt [31]. CRDTs beseitigen jedoch weder jede fachliche Mehrdeutigkeit noch die Notwendigkeit eines geeigneten Datenmodells. Bei einem konkurrierend veränderten fachlichen Wert muss die Anwendung priorisieren, zusammenführen oder der nutzenden Person eine Entscheidung vorlegen. Für eine Anwendung ohne Synchronisation sind weder CRDTs noch Operational Transformation erforderlich.

2.4.2 Lokale Datenhaltung und Offline-First

Die zentrale technische Konsequenz des Paradigmas ist die lokale Primärpersistenz. Die Anwendung muss die für ihre Kernfunktionen erforderlichen Daten dauerhaft auf dem Endgerät speichern und lokal lesen und schreiben können. Ein flüchtiger UI-Zustand, ein Browser-Cache oder die optimistische Darstellung einer unbestätigten Serveränderung genügen dafür nicht.

Offline-First und Local First sind eng verwandte, aber nicht deckungsgleiche Begriffe. Für die vorliegende Arbeit bezeichnet Offline-First die Anforderung, dass die Anwendung auch ohne Verbindung funktionsfähig bleibt und Netzwerkzugriffe nicht den primären Interaktionspfad bilden. Local First umfasst zusätzlich, wem die Daten unterstehen, wie sie langfristig erhalten werden und wie mehrere lokale Kopien zusammenarbeiten können. Eine Anwendung kann daher offlinefähig sein, ohne ihren Datenbestand in kontrollierbarer Form bereitzustellen. Umgekehrt kann eine lokale Anwendung die Datenhoheit stärken, ohne zu synchronisieren.

Für die lokale relationale Datenhaltung eignet sich SQLite als eingebettete Datenbank. SQLite arbeitet als serverlose, konfigurationsarme Bibliothek innerhalb des Anwendungsprozesses, speichert eine relationale Datenbank in einer Datei und unterstützt ACID-Transaktionen [32]. Zusammengehörige Änderungen können dadurch atomar verarbeitet und bei einem Fehler zurückgerollt werden. Diese Eigenschaften betreffen die Datenbank-Engine und sind nicht mit einer vollständigen Backup- oder Wiederherstellungsgarantie gleichzusetzen.

Im Zielartefakt ist die schreibbare SQLite-Datei mit den Benutzerdaten vom unveränderlichen Installationsverzeichnis zu unterscheiden. Die Datenbankdatei liegt in einem Benutzerdatenpfad und wird bei Updates nicht überschrieben, während das Installationsverzeichnis unveränderliche Programmressourcen enthält. Die Speicherung in einer Datei vereinfacht Kopieren, Export und Backup, schützt aber nicht automatisch vor Dateibeschädigung, versehentlichem Löschen, Schadsoftware oder dem Verlust

des Endgeräts. Backups müssen deshalb konsistent erstellt, gegen unberechtigten Zugriff geschützt und wiederherstellbar sein. Ebenso ist zu entscheiden, ob sensible Inhalte unverschlüsselt abgelegt werden können oder Verschlüsselungs- und Schlüsselverwaltungsmechanismen erforderlich sind.

Eine relationale lokale Datenbank benötigt ein explizit verwaltetes Schema. Bei Weiterentwicklung müssen bestehende Datenbestände überführt werden, sodass ein leerer Erstinstallationsstand nicht der einzige relevante Fall ist. Schema-Migrationen beschreiben diese Entwicklung als versionierte, reproduzierbare Änderungen. Dabei sind Reihenfolge, Abhängigkeiten, Default-Werte, Datenübernahmen und die Wiederholbarkeit der Schritte zu berücksichtigen. Migrationen sollten innerhalb geeigneter transaktionaler Grenzen ausgeführt werden, damit ein Fehler nicht zu einem teilweise aktualisierten Schema führt. Ob der eingesetzte Migration Runner dies für jeden Schritt leistet, ist produkt- und konfigurationsabhängig und muss für das Zielartefakt geprüft werden.

Drizzle kann die Umsetzung unterstützen, indem Datenbankschemata in TypeScript beschrieben und SQL-Migrationen erzeugt beziehungsweise angewendet werden [33]. Ein typgesichertes ORM ersetzt jedoch nicht die fachliche Prüfung der erzeugten SQL-Änderungen. Für das Zielartefakt wird daher gefordert, dass Migrationen mit dem Anwendungspaket ausgeliefert, beim Start kontrolliert angewendet und für alle unterstützten Ausgangsversionen getestet werden. Die Anwendung muss erkennen, welche Migrationen bereits ausgeführt wurden, und darf einen bestehenden Datenbestand nicht grundlos durch eine leere Datenbank ersetzen.

Für die Fallstudie ist dies relevant, weil die Ablösung der cloudbasierten Persistenz mehr als einen Datenbanktreiberwechsel bedeutet. Sie erfordert ein lokales relationales Modell, die Übernahme des bestehenden Datenbestands und einen kontrollierten Lebenszyklus für spätere Schemaänderungen.

Eine Local-First-Anwendung kann unterschiedliche Synchronisationsstrategien verwenden. Im Einzelplatzszenario ist keine Synchronisation erforderlich, da die lokale Datenbank die autoritative Persistenz bleibt und durch Backups ergänzt wird. Bei mehreren Geräten kann ein Dienst als sekundäre Ablage oder eine Synchronisation über lokales Netzwerk beziehungsweise Peer-to-Peer eingesetzt werden. Die Wahl hängt von Geräteanzahl, Kollaboration, Datenschutz, Verfügbarkeit und Wiederherstellbarkeit ab. Synchronisation ist nicht mit einer bloßen Kopie der Datenbank gleichzusetzen. Änderungen müssen erkannt, übertragen und zusammengeführt werden. Ein Last-Write-Wins-Verfahren reduziert technische Konflikte, kann fachlich relevante Änderungen aber unbemerkt überschreiben. Bei relationalen Daten ist deshalb die Synchronisationsebene, etwa Datensatz, Attribut oder fachliche Operation, ausdrücklich festzulegen. Ob eine Mehrgeräte- oder Mehrpersonensynchronisation zum Zielartefakt gehört, wird in der Fallstudie begründet. Das Paradigma schreibt sie nicht zwingend vor.

2.4.3 Abgrenzung zu cloudzentrierten und traditionellen Desktop-Anwendungen

Die folgende Abgrenzung bezieht sich auf cloudzentrierte beziehungsweise serverabhängige Webanwendungen und nicht auf sämtliche Webarchitekturen. Der weitergehende Begriff *Cloud-Native* bezeichnet einen eigenständigen Architektur- und Entwicklungsansatz [34] und wird hier nicht als Bezeichnung der Ausgangsarchitektur verwendet.

Auf der Artefaktebene liegt in der betrachteten cloudzentrierten Web-Referenzarchitektur die dauerhaft maßgebliche Datenbasis im Backend. Der Client verwaltet UI-Zustand und gegebenenfalls einen Cache, während fachliche Änderungen an den Server übermittelt und dort persistiert werden. Vorteile sind zentrale Verwaltung, einfache Bereitstellung neuer Versionen, eine gemeinsame Datenbasis und geräteübergreifende Zusammenarbeit. Dem stehen Netzwerk- und Anbieterabhängigkeit, zentrale Angriffsziele und eine eingeschränkte Kontrolle über die gespeicherten Daten gegenüber.

Eine traditionelle Desktop-Anwendung führt Programm und Daten überwiegend lokal aus, funktioniert im vorgesehenen Nutzungsszenario ohne Netzwerk und greift unmittelbar auf lokale Dateien oder eine eingebettete Datenbank zu. Sie ist jedoch nicht allein dadurch local-first. Für die Einordnung sind zusätzlich die lokale Autorität der Daten, ihre Kontrollierbarkeit und gegebenenfalls die Fähigkeit zur Synchronisation mit weiteren Kopien relevant.

Local First verbindet als heuristische Einordnung die unmittelbare Interaktion und Datenkontrolle einer lokalen Anwendung mit ausgewählten Netzwerkdiensten wie Synchronisation, Backup und Kollaboration. Diese Dienste sind optional und können im Einzelplatzszenario vollständig entfallen. Der entscheidende Unterschied zur cloudzentrierten Architektur liegt in der Reihenfolge und Autorität der Operationen. Die lokale Kopie ist primär, während ein Server nur eine ergänzende Funktion übernimmt.

Für das Transformationsszenario bedeutet dies, dass die Wahl von Electron allein noch keine Local-First-Architektur erzeugt. Electron stellt die Ausführungsumgebung und technische Mechanismen für eine kontrollierte Vermittlung zwischen Web-UI und lokalen Ressourcen bereit. Die wirksame Kontrolle entsteht erst durch Preload- und IPC-Design, Eingabevalidierung und begrenzte Berechtigungen. Erst die Kombination aus lokaler Datenbank, lokalem Datenzugriff, offlinefähiger Geschäftslogik und einem bewusst optionalen Netzwerkpfad verwirklicht die für diese Arbeit relevanten Local-First-Eigenschaften.

2.4.4 Relevanz für das Transformationsszenario

Für die Fallstudie wird zunächst ein lokales Einzelplatzszenario betrachtet. Die Kernfunktionen des Studienplaners sollen ohne Netzwerkverbindung verfügbar sein. Eine Mehrgeräte- oder Mehrpersonensynchronisation ist für dieses Zielbild kein notwendiger Bestandteil, kann aber durch eine geeignete Datenhaltung später ergänzt werden. Diese Scope-Entscheidung ermöglicht es, lokale Primärpersistenz und Offlinefähigkeit zu untersuchen, ohne bereits die zusätzliche Komplexität verteilter Konfliktauflösung als notwendige Voraussetzung einzuführen.

Die Überführung des KI-generierten Web-Prototyps verändert nicht nur den Speicherort der Daten, sondern die Autorität der Datenquelle, die Ausführungsreihenfolge der Geschäftsoperationen und die Verantwortung für Datenintegrität, Migration, Backup und Wiederherstellung. Aus einer Anfrage an einen entfernten Dienst wird eine lokale Transaktion, die innerhalb der Anwendung ausgeführt und dauerhaft in SQLite gespeichert wird. Der Netzwerkzugriff wird dadurch zur optionalen Erweiterung, sofern keine Funktion ausdrücklich auf einen externen Dienst angewiesen ist.

Für die Transformation ergeben sich mehrere architektonische Verschiebungen. Erstens ersetzt eine lokale Primärpersistenz die entfernte autoritative Persistenz. Zweitens bildet eine Datenzugriffsschicht fachliche Operationen auf das lokale relationale Modell ab, statt die Benutzeroberfläche direkt mit HTTP- oder BaaS-Aufrufen zu koppeln. Drittens werden Transaktionen und Integritätsbedingungen lokal nachvollziehbar umgesetzt. Viertens ist zu klären, ob Identitäts-, Berechtigungs- und Mandantenkonzepte aus dem Cloud-System für das lokale Nutzungsszenario weiterhin erforderlich sind. Fünftens sind Migrationen, Backups und Wiederherstellung als Bestandteile des Anwendungslifecycles zu behandeln.

Auch die Benutzeroberfläche muss an die neue Zustandssemantik angepasst werden. Ein Ladezustand, der auf die Antwort eines Servers wartet, ist bei einer lokalen Transaktion nicht mehr der primäre Interaktionsablauf. Stattdessen sind lokale Erfolge, Validierungsfehler und gegebenenfalls ausstehende Synchronisationsvorgänge zu unterscheiden. Optimistische UI-Aktualisierungen lassen sich bei einer lokalen Datenbanktransaktion teilweise durch unmittelbare lokale Persistierung ersetzen, bleiben jedoch bei einem späteren Abgleich mit weiteren Geräten oder einem Dienst relevant.

In der sicherheitsgehärteten Zielarchitektur wird die lokale Persistenz nicht direkt aus dem Renderer-Prozess angesprochen. Dies ist eine bewusste Designentscheidung der Zielarchitektur und keine automatische Eigenschaft jeder Electron-Anwendung. Der Renderer verarbeitet weiterhin Web-Code und erhält keinen allgemeinen Datei- oder Datenbankzugriff. Fachlich begrenzte Operationen werden über die in Abschnitt 2.2.3 beschriebene Preload- und IPC-Schnittstelle an den privilegierten Anwendungsteil vermittelt. Dort werden Eingaben validiert und auf erlaubte lokale Operationen begrenzt. Damit verbindet das Local-First-Zielbild die Datenarchitektur mit der Sicherheitsarchitektur der Electron-Anwendung.

Die lokale Ausführung erweitert zugleich die Verantwortung für Sicherheit und Betrieb. Die Anwendung muss Datenbanken kontrolliert anlegen und aktualisieren, Fehler bei Schreibvorgängen behandeln, Backups ermöglichen und mit beschädigten oder nicht unterstützten Datenbankständen umgehen. Bei einer späteren Synchronisation kämen Konfliktregeln, Authentifizierung, Verschlüsselung und die Wiederholbarkeit von Änderungen hinzu. Die Transformation reduziert die unmittelbare Abhängigkeit vom Cloud-Anbieter, beseitigt aber nicht die Komplexität der Datenhaltung. Sie verlagert diese Komplexität in die lokale Anwendung, wo sie explizit modelliert, getestet und abgesichert werden muss.

Daraus lassen sich für die spätere Analyse und Evaluation überprüfbare Kriterien ableiten. Die zentralen fachlichen Funktionen müssen ohne Netzwerk dauerhaft verfügbar sein. Lokale Schreibvorgänge müssen dauerhaft persistiert werden. Vorhandene Datenbestände müssen nach der Migration wiederverwendbar bleiben. Nicht unterstützte Datenbankstände müssen kontrolliert behandelt werden. Backup- und Wiederherstellungsprozesse müssen nachvollziehbar sein. Die Sicherheitsanforderungen aus Abschnitt 2.3.3 müssen durch Konfigurationsprüfungen, statische Analysen und geeignete Tests überprüft werden.

Das Local-First-Paradigma liefert damit ein architektonisches Zielbild und zugleich Bewertungskriterien für die Transformation. Eine erfolgreiche Überführung besteht nicht darin, das Web-Frontend als installierbare Datei zu verpacken. Erforderlich ist eine lokale Daten- und Ausführungsarchitektur, in der die zentralen fachlichen Funktionen ohne Netzwerkverbindung verfügbar sind, Daten kontrollierbar gespeichert werden und spätere Erweiterungen wie Synchronisation nicht durch unklare Datenhaltung verhindert werden. Die daraus entstehenden Sicherheitsanforderungen an lokale Daten, IPC, Dateisystemzugriffe und Abhängigkeiten bilden die Grundlage für die Analyse und den Entwurf der Zielarchitektur in den folgenden Kapiteln.

Kapitel 3

Methodisches Vorgehen

Kapitel 4

Analyse und Problemanalyse

Kapitel 5

Entwurf und Entwicklung des Artefakts

Kapitel 6

Evaluation und Demonstration

Kapitel 7

Diskussion

Kapitel 8

Fazit und Ausblick

Abbildungsverzeichnis

Quellcodeverzeichnis

Tabellenverzeichnis

2.1	Vergleich KI-gestützter Entwicklungswerkzeuge nach Zielsetzung, Technologie-Stack und Architektur-Charakter	7
-----	--	---

Literaturverzeichnis

- [1] Wikipedia contributors, „Vibe Coding,” Wikipedia, The Free Encyclopedia, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Vibe_coding (Zitiert auf Seite 6.)
- [2] B. Edwards, „Will the Future of Software Development Run on Vibes?” Ars Technica, 2025. [Online]. Available: <https://arstechnica.com/ai/2025/03/is-vibe-coding-with-ai-gnarly-or-reckless-maybe-some-of-both/> (Zitiert auf Seite 6.)
- [3] Replit, „Replit Agent – Documentation,” 2026. [Online]. Available: <https://docs.replit.com/replitai/agent> (Zitiert auf Seite 6.)
- [4] Lovable, „Welcome to Lovable – Documentation,” 2026. [Online]. Available: <https://docs.lovable.dev/introduction/welcome> (Zitiert auf Seiten 6 und 10.)
- [5] Figma, „Figma Make – Prompt to Code,” 2026. [Online]. Available: <https://www.figma.com/make/> (Zitiert auf Seite 6.)
- [6] ISO/IEC, „ISO/IEC 25010:2023 – Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – Product Quality Model,” International Organization for Standardization, 2023. [Online]. Available: <https://www.iso.org/standard/78176.html> (Zitiert auf Seite 7.)
- [7] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, und R. Karri, „Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022. [Online]. Available: <https://arxiv.org/abs/2108.09293> (Zitiert auf Seiten 7 und 16.)
- [8] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, und L. C. Cordeiro, „How Secure is AI-Generated Code: A Large-Scale Comparison of Large Language Models,” *Empirical Software Engineering*, 2024. [Online]. Available: <https://arxiv.org/abs/2404.18353> (Zitiert auf Seiten 7 und 16.)
- [9] O. Asare, M. Nagappan, und N. Asokan, „Is GitHub’s Copilot as Bad as Humans at Introducing Vulnerabilities in Code?” *Empirical Software Engineering*, 2023. [Online]. Available: <https://arxiv.org/abs/2204.04741> (Zitiert auf Seiten 7 und 16.)
- [10] N. Perry, M. Srivastava, D. Kumar, und D. Boneh, „Do Users Write More Insecure Code with AI Assistants?” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, S. 2785–2799. [Online]. Available: <https://arxiv.org/abs/2211.03622> (Zitiert auf Seiten 7 und 16.)
- [11] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, und M. Jadhwal, „We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs,” in *34th USENIX Security Symposium*, 2025. [Online]. Available: <https://arxiv.org/abs/2406.10279> (Zitiert auf Seiten 8 und 17.)
- [12] GitClear, „Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality,” GitClear, Tech. Rep., 2024. [Online]. Available: https://www.gitclear.com/coding_on_copilot_data_shows_ais_downward_pressure_on_code_quality (Zitiert auf Seite 8.)
- [13] J. Becker, N. Rush, E. Barnes, und D. Rein, „Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” METR, Tech. Rep., 2025. [Online]. Available: <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/> (Zitiert auf Seite 8.)
- [14] R. Albergotti, „The Hottest New Vibe Coding Startup May Be a Sitting Duck for Hackers,” Semafor, 2025. [Online]. Available: <https://www.semafor.com/article/05/29/2025/the-hottest-new-vibe-coding-startup-lovable-is-a-sitting-duck-for-hackers> (Zitiert auf Seiten 8 und 10.)

- [15] S. Sharwood, „Vibe Coding Service Replit Deleted User’s Production Database, Faked Data, Told Fibs Galore,” The Register, 2025. [Online]. Available: https://www.theregister.com/2025/07/21/replit_saastr_vibe_coding_incident/ (Zitiert auf Seite 8.)
- [16] MDN, „Client-server overview – Learn web development,” MDN Web Docs, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Client-Server_overview (Zitiert auf Seite 9.)
- [17] —, „JavaScript frameworks and libraries – Learn web development,” MDN Web Docs, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction (Zitiert auf Seite 9.)
- [18] —, „Overview of HTTP,” MDN Web Docs, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview> (Zitiert auf Seite 9.)
- [19] Wikipedia contributors, „Thin Client,” Wikipedia, The Free Encyclopedia, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Thin_client (Zitiert auf Seite 9.)
- [20] Electron contributors, „Process Model,” Electron Documentation, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/process-model> (Zitiert auf Seiten 11 und 13.)
- [21] —, „Security,” Electron Documentation, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/security> (Zitiert auf Seiten 12, 13, 14 und 15.)
- [22] —, „Inter-Process Communication,” Electron Documentation, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/ipc> (Zitiert auf Seiten 12 und 14.)
- [23] Tauri contributors, „What is Tauri?” Tauri Documentation, 2026. [Online]. Available: <https://v2.tauri.app/start/> (Zitiert auf Seite 12.)
- [24] tw93, „Pake – Turn any webpage into a desktop app with one command,” GitHub, 2026. [Online]. Available: <https://github.com/tw93/Pake> (Zitiert auf Seite 12.)
- [25] NWJS Project, „JavaScript Contexts in NWJS,” NW.js Documentation, 2026. [Online]. Available: <https://nwjs.readthedocs.io/en/latest/> (Zitiert auf Seite 12.)
- [26] Flutter contributors, „Desktop support for Flutter,” Flutter Documentation, 2026. [Online]. Available: <https://docs.flutter.dev/platform-integration/desktop> (Zitiert auf Seite 12.)
- [27] Electron contributors, „Context Isolation,” Electron Documentation, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/context-isolation> (Zitiert auf Seite 14.)
- [28] —, „Process Sandboxing,” Electron Documentation, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/tutorial/sandbox> (Zitiert auf Seite 14.)
- [29] M. Souppaya und K. Scarfone, „Guide to Data-Centric System Threat Modeling,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-154, 2016. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/154/ipd> (Zitiert auf Seite 16.)
- [30] OWASP Foundation, „OWASP Top 10: 2025,” OWASP, 2025. [Online]. Available: <https://owasp.org/Top10/2025/> (Zitiert auf Seite 17.)
- [31] M. Kleppmann, A. Wiggins, P. van Hardenberg, und M. McGranaghan, „Local-first software: You own your data, in spite of the cloud,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. Association for Computing Machinery, 2019, S. 154–178. [Online]. Available: <https://dl.acm.org/doi/10.1145/3359591.3359737> (Zitiert auf Seiten 18 und 19.)
- [32] SQLite Consortium, „About SQLite,” SQLite Documentation, 2025. [Online]. Available: <https://www.sqlite.org/about.html> (Zitiert auf Seite 19.)
- [33] Drizzle Team, „Drizzle ORM – Overview and migrations,” Drizzle Documentation, 2026. [Online]. Available: <https://orm.drizzle.team/docs/overview> (Zitiert auf Seite 20.)
- [34] Cloud Native Computing Foundation, „CNCF Cloud Native Definition v1.1,” CNCF Technical Oversight Committee, 2024. [Online]. Available: <https://github.com/cncf/toc/blob/main/DEFINITION.md> (Zitiert auf Seite 20.)

Anhang A

Supplemental Information

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc,

molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.